Verified.AI: GenAI Powered Claim Verification with Retrieval-Augmented Evidence

Alice Ji
College of William & Mary
Virginia, USA
axji@wm.edu

Camly Tran
College of William & Mary
Virginia, USA
cttran@wm.edu

May 12, 2026

Abstract

The rapid spread of misinformation across digital platforms has created a need for fact-checking systems that are both automated and evidence-grounded. This project implements Verified.AI, a Retrieval-Augmented Generation (RAG) style claim verification system that retrieves Wikipedia-based evidence and classifies natural language claims as `SUPPORTS`, `REFUTES`, or `NOT ENOUGH INFO`. Unlike standalone language-model responses, the system emphasizes transparency by returning not only a predicted label, but also verifier confidence scores and ranked evidence passages used to support the prediction.

The implemented pipeline combines sentence-level evidence retrieval, FAISS-based dense vector search, CrossEncoder reranking, and a DeBERTa-based natural language inference verifier. Using the FEVER dataset, we constructed a targeted Wikipedia evidence subset, built retriever-specific FAISS indexes, and evaluated the system using Recall@K and final classification accuracy. On a targeted 100-example FEVER evaluation, the dense retrieval plus reranker pipeline achieved Recall@1 of 0.8133, Recall@5 of 0.9467, and Recall@10 of 0.9733. The full retrieval, reranking, and verification pipeline achieved a final accuracy of 0.7733 over 75 examples with usable gold evidence.

In addition to the backend pipeline, the project includes a FastAPI backend and React frontend that allow users to submit claims, configure retrieval settings, view predicted verdicts, inspect verifier scores, and examine ranked evidence trails. The interface supports a controlled FEVER-based retrieval mode for evaluation as well as an experimental Live Wikipedia mode for interactive demonstration. Overall, the project demonstrates that retrieval-augmented claim verification can provide a more interpretable alternative to ungrounded generative responses, while also revealing that the main remaining bottleneck lies in verifier reasoning rather than evidence retrieval.

1 Introduction

The rapid expansion of digital information has made it increasingly difficult for users to determine whether online claims are accurate, misleading, or unsupported. Social media platforms, search engines, and generative AI systems allow information to spread quickly, but they do not always make the verification process transparent. Even when a system provides a fluent answer or a short list of sources, users often cannot see how evidence was retrieved, ranked, interpreted, or connected to the final claim-level judgment. This creates a need for tools that do more than simply return an answer: a useful claim-verification system should expose the evidence pipeline behind the prediction.

Large language models have demonstrated strong performance on many natural language tasks, but they can still generate plausible responses that are not fully grounded in verifiable evidence. Search engines and AI-enhanced search tools increasingly surface source-backed summaries, but these systems generally hide many of the underlying retrieval and ranking decisions from the user. Traditional machine learning classifiers can also be limited because they often predict labels without showing which evidence influenced the decision. These limitations motivate a retrieval-augmented approach where the system first retrieves evidence, then performs evidence-conditioned reasoning, while preserving visibility into each step of the process.

This project implements *Verified.AI*, a Retrieval-Augmented Generation (RAG) style claim-verification workbench. Given a natural language claim, the system retrieves relevant Wikipedia-based evidence, optionally reranks candidate evidence passages, and uses a natural language inference verifier to classify the claim as `SUPPORTS`, `REFUTES`, or `NOT ENOUGH INFO`. The goal is not to position the system as a complete replacement for Google Search, commercial AI assistants, or professional fact-checking organizations. Instead, Verified.AI is designed as an interpretable and configurable environment for studying how retrieval settings, reranking, evidence sources, and verifier behavior affect claim verification outcomes.

The core evaluation uses the FEVER dataset, a benchmark for fact extraction and verification. FEVER provides claims, labels, and sentence-level Wikipedia evidence annotations, making it useful for evaluating both evidence retrieval and final claim classification. In our implementation, FEVER claims and gold evidence were normalized into structured page and sentence identifiers. A targeted Wikipedia subset was then constructed from relevant evidence pages, embedded using sentence-transformer models, indexed with FAISS, and evaluated using Recall@K. Retrieved evidence was then reranked with a CrossEncoder and passed into a DeBERTa-based natural language inference verifier.

A major design goal of the system is configurability. The web interface allows users to submit claims, switch between FEVER-based and Live Wikipedia retrieval modes, adjust retrieval volume, enable or disable CrossEncoder reranking, select available retriever models, inspect verifier scores, and view ranked evidence trails. This makes the system useful not only for producing a prediction, but also for comparing how different retrieval and verification settings influence the final result. The reported quantitative results are based on the controlled FEVER mode, while Live Wikipedia mode serves as an experimental extension for interactive demonstration with current Wikipedia content.

This project therefore contributes both a modular technical pipeline and a user-facing verification interface. The final system shows that retrieval-augmented fact verification can produce interpretable claim-level predictions with strong evidence retrieval performance in a controlled FEVER setting. At the same time, the results reveal that retrieval alone is not enough: most remaining errors come from the verifier misinterpreting retrieved evidence, being too conservative, or struggling with indirect and multi-hop reasoning. These findings position Verified.AI as a practical case study in building transparent, configurable, and evidence-grounded AI systems rather than a universal fact-checking solution.

2 Market Analysis

2.1 Target Segment

Verified.AI targets users who need more transparency than a standard search result or conversational answer provides. The primary audience includes students, researchers, journalists, educators, and developers who want to inspect how a claim-verification system retrieves evidence and arrives

at a prediction. Rather than focusing only on end users who want a single fact-checking answer, the system is especially suited for users who want to compare retrieval settings, examine ranked evidence, and understand how different model choices affect the final verdict.

A second target segment is software engineering and AI education. Because the system exposes multiple stages of a retrieval-augmented pipeline, it can serve as a teaching and experimentation tool for understanding dense retrieval, reranking, natural language inference, and evidence-grounded prediction. Users can adjust retrieval volume, toggle reranking, switch evidence sources, and compare available retrievers. This makes the platform useful as a lightweight claim-verification workbench rather than only as a consumer-facing fact checker.

2.2 Problem Justification

The growth of online information has made claim verification more difficult, especially when claims are spread without context or supporting evidence. Existing tools can often provide relevant sources, but they do not always reveal how those sources were selected or how strongly they support a specific claim. For example, a search engine may return a list of pages, and an AI assistant may summarize information from multiple sources, but users typically cannot inspect the full retrieval and verification process behind the answer.

This project addresses that gap by making the intermediate stages of claim verification visible. The system does not only return a label such as `SUPPORTS` or `REFUTES`; it also displays retrieved evidence, similarity scores, rerank scores, verifier scores, and configuration settings. This matters because claim verification is not only a prediction problem, but also an interpretability problem. If a user cannot see what evidence was used, then the prediction is difficult to trust, even if the final label is correct.

The FEVER dataset provides a controlled setting for developing and evaluating this type of system because it includes claims, labels, and sentence-level evidence annotations from Wikipedia. This allows retrieval quality to be measured directly using Recall@K and allows the final verifier to be evaluated against known labels. At the same time, the project acknowledges that FEVER-style claims are different from arbitrary real-world claims. For that reason, the interface also includes an experimental Live Wikipedia mode, while the reported metrics remain tied to the controlled FEVER evaluation.

2.3 Competitors

Verified.AI exists in a space that overlaps with search engines, AI assistants, and fact-checking platforms, but it differs from each in its primary purpose. Modern search engines such as Google increasingly provide AI-generated summaries and source links. These tools are useful for general information access, but they are optimized for broad search rather than transparent experimentation with retrieval and verification settings. Users generally cannot choose the embedding model, adjust retrieval depth, disable reranking, or inspect verifier score behavior.

Conversational AI assistants such as ChatGPT and Claude can answer factual questions and often provide explanations, but their reasoning process and retrieval pipeline are mostly hidden from the user. Even when sources are available, the user typically sees the final answer rather than the full evidence-ranking and verifier-scoring process. Verified.AI differs by exposing these intermediate steps and allowing users to compare how different settings affect results.

Professional fact-checking organizations provide higher-quality human-reviewed verification, but their workflows are manual and not designed for interactive experimentation. Dedicated fact-checking tools and research systems may provide automated claim verification, but many are built

as backend models or benchmark systems rather than user-facing educational interfaces. Verified.AI occupies a middle ground: it is not intended to replace professional fact-checking or broad web search, but to provide a transparent, configurable environment for studying retrieval-augmented verification.

2.4 Monetization and Go-to-Market

The current version of Verified.AI is best positioned as a free academic and educational tool. Its immediate go-to-market path would be classroom use, research demos, and open-source adoption by students or developers interested in retrieval-augmented AI systems. The platform is lightweight enough to run locally and modular enough for others to modify retrieval, reranking, or verification components.

A plausible future monetization path would be a freemium or hosted research-tool model. The free version could support local FEVER-style experiments and limited Live Wikipedia retrieval, while a hosted version could provide larger indexes, more stable live retrieval, persistent projects, API access, and team-based evaluation dashboards. Another possible model is an API service for developers who want evidence-backed claim verification in their own applications.

However, the strongest near-term value is not direct competition with commercial search or AI assistants. Instead, Verified.AI's main differentiator is transparency and configurability. The system gives users control over retrieval settings and exposes the evidence trail behind each prediction. This makes it valuable as a demonstration platform, teaching tool, and foundation for future research into interpretable retrieval-augmented verification.

3 System Overview

Verified.AI is implemented as an interactive claim-verification workbench. The system allows a user to enter a natural language claim, choose a retrieval source, configure retrieval settings, and inspect the evidence used to support the final prediction. The main purpose of the interface is not only to return a claim label, but also to make the retrieval and verification process visible to the user.

The main verification page contains a claim input box, a source toggle, and a verification result panel. Users can choose between *FEVER Gold* mode and *Live Wiki* mode. FEVER Gold mode uses the controlled FEVER-based retrieval pipeline that was used for evaluation. Live Wiki mode is an experimental extension that retrieves evidence from current Wikipedia pages using live API requests. This distinction is important because the reported quantitative results in this paper are based on FEVER Gold mode, while Live Wiki mode is intended primarily for interactive demonstration.

After a claim is submitted, the system returns a predicted label: `SUPPORTS`, `REFUTES`, or `NOT ENOUGH INFO`. The result panel also displays a confidence value and the verifier scores for entailment, contradiction, and neutral. These values help users see whether the verifier strongly favored one label or whether the decision was uncertain. This is especially useful for borderline cases where retrieved evidence is relevant but not strong enough to clearly support or refute the claim.

A central feature of the interface is the evidence trail. Retrieved evidence is shown as a ranked list of evidence cards. Each card includes the source page, sentence text, similarity score, optional rerank score, and sentence identifier. This allows the user to inspect whether the model is relying on meaningful evidence or merely retrieving topically related but insufficient passages. By exposing this ranked evidence, the system makes the verification process more interpretable than a black-box classifier or a single generated answer.

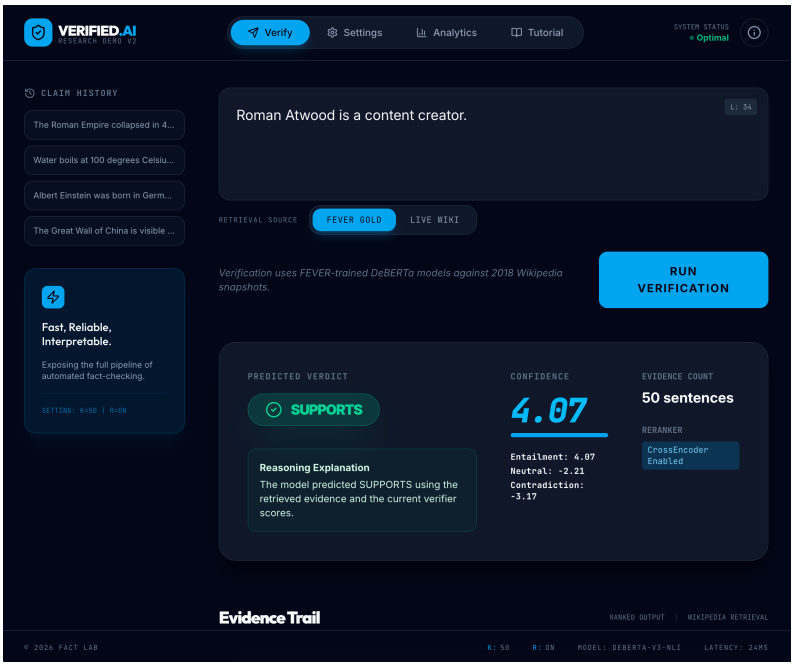

Figure 1: Main verification interface showing the claim input, source toggle, predicted verdict, confidence score, verifier score breakdown, and evidence count.

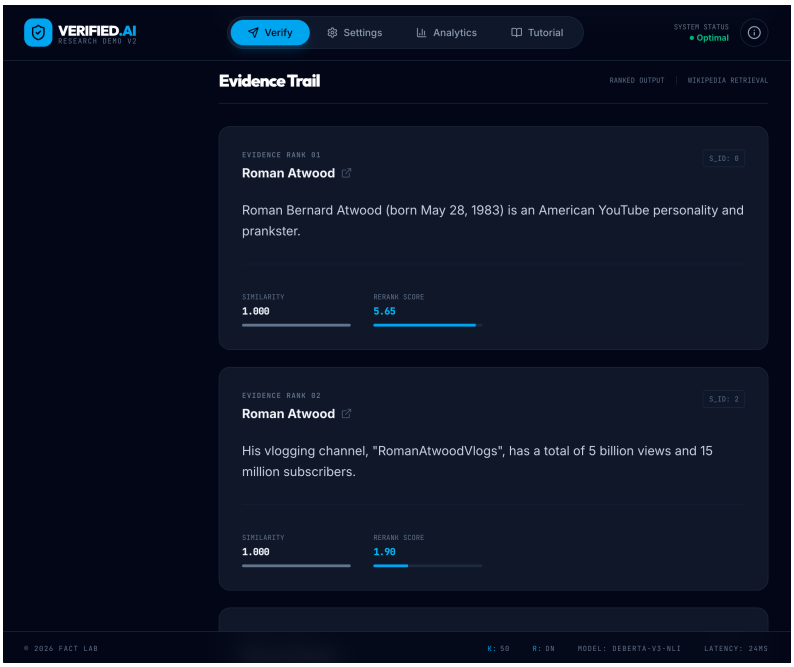

Figure 2: Evidence trail showing ranked retrieved evidence passages with source page, sentence text, similarity score, rerank score, and sentence identifier.

Verified.AI also includes an advanced settings page that exposes key retrieval configuration options. Users can adjust the retrieval volume, enable or disable CrossEncoder reranking, and select among supported retriever configurations. In the evaluated FEVER pipeline, MiniLM is the

stable retriever used for reported metrics. Additional retriever options, such as BGE and E5, are included as experimental settings in the interface to support comparison and future extension. This settings page reinforces the system's role as a configurable verification workbench rather than a fixed one-model fact checker.

The interface also contains an analytics dashboard. This dashboard summarizes system-level behavior using retrieval and verification metrics such as Recall@1, Recall@5, average pipeline accuracy, NOT ENOUGH INFO prediction rate, and label confidence distributions. The dashboard is useful for comparing different retrieval configurations and for understanding where the system succeeds or fails. Rather than presenting the model verdict as final and unquestionable, the analytics page encourages users to evaluate the behavior of the pipeline as a whole.

Finally, the application includes a tutorial page that explains the verification pipeline step by step. The tutorial walks through claim input, dense retrieval, reranking, and verification. This page is designed to make the system accessible to users who may not already understand retrieval-augmented generation or natural language inference. By combining the tutorial, settings page, evidence trail, and analytics dashboard, Verified.AI functions as both a claim-verification interface and an educational tool for exploring retrieval-augmented AI systems.

4 Technical Implementation

Verified.AI is implemented as a modular retrieval-augmented claim verification system. The system is divided into data preprocessing, targeted corpus construction, dense retrieval, reranking, verification, and user interface layers. This modular structure was important because each stage could be tested independently before being connected into the final application.

4.1 FEVER Preprocessing

The first stage of the system processes the FEVER dataset and Wikipedia evidence corpus. FEVER examples are stored in JSONL format, where each example contains a claim, a label, and one or more evidence annotations. The raw evidence annotations use a nested structure that includes page names and sentence identifiers. To make the data easier to use during retrieval and evaluation, the preprocessing step normalizes each example into a simplified structure containing the claim, gold label, and clean evidence references of the form `(page, sentence_id)`.

The Wikipedia dump is processed separately. Each Wikipedia page record contains an identifier, full page text, and a `lines` field containing numbered sentence-level entries. The preprocessing script parses these lines and creates a sentence-level retrieval corpus. Each sentence is saved with its Wikipedia page name, sentence identifier, and text. This structure is necessary because retrieval evaluation compares retrieved results against FEVER's gold evidence annotations using exact page and sentence identifiers.

4.2 Targeted Evidence Subset

Because the full FEVER Wikipedia corpus contains over 25 million sentence entries, indexing the entire corpus locally was not practical for the project timeline and hardware constraints. Instead, the system uses a targeted subset for controlled evaluation. This subset is built by collecting the Wikipedia page names that appear in the gold evidence annotations for the selected FEVER examples. The full sentence corpus is then filtered to keep only sentences from those target pages.

This targeted setup allows the system to evaluate whether the retriever can locate the correct sentence when the relevant evidence page is available in the candidate corpus. It also makes the

evaluation computationally manageable while preserving sentence-level evidence matching. The reported results use a targeted subset built from the first 100 FEVER examples. After removing examples without usable gold evidence, 75 examples were evaluated.

4.3 FAISS Index Construction

After the targeted corpus is built, each evidence sentence is embedded into a dense vector representation. The main evaluated retriever uses `sentence-transformers/all-MiniLM-L6-v2`. The resulting sentence embeddings are converted to `float32`, normalized with NumPy, and stored in a FAISS `IndexFlatIP` index. Since the embeddings are normalized, inner product similarity approximates cosine similarity.

During development, FAISS normalization caused local native crashes on macOS, so the implementation avoids `faiss.normalize_L2()` and instead performs normalization manually with NumPy. This made the index-building process more stable. The FAISS index is saved to disk along with a metadata file that maps FAISS row identifiers back to the original page name, sentence identifier, and sentence text. This metadata is essential because FAISS returns numeric indices, while the frontend and evaluation code need interpretable evidence passages.

The backend was also designed to support multiple retriever indexes, including MiniLM, BGE, and E5. However, the reported evaluation uses MiniLM because it was the stable retriever used throughout the controlled FEVER evaluation. Additional retriever options were added to support experimentation through the UI.

4.4 Dense Retrieval

At inference time, a user claim is embedded using the selected retriever model. The claim embedding is normalized and searched against the corresponding FAISS index. The system retrieves a

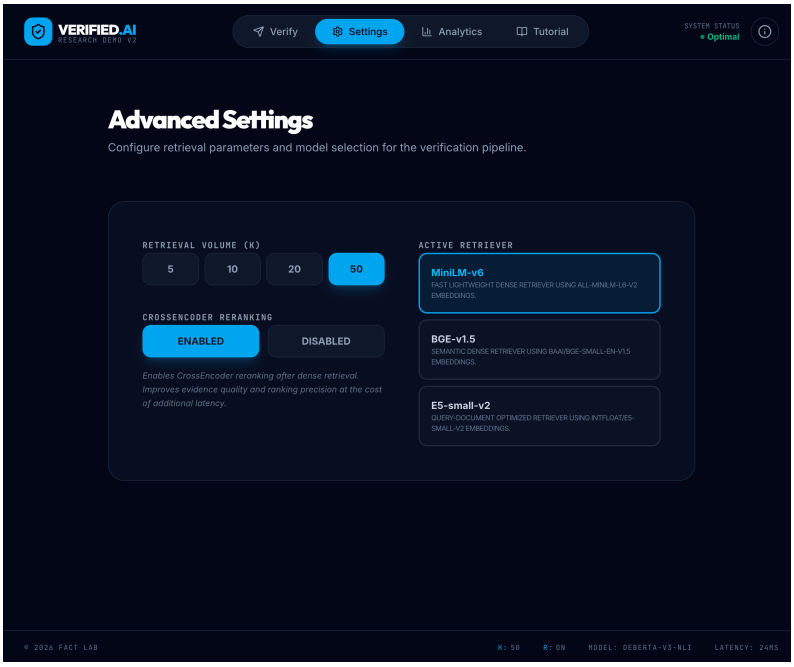

Figure 3: Advanced settings panel allowing users to adjust retrieval volume, toggle CrossEncoder reranking, and choose the active retriever configuration.

configurable number of candidate evidence sentences, such as the top 50 candidates, before any reranking is applied.

For retrievers that require special text formatting, the system applies the appropriate query or passage prefixes. For example, E5-style models use `query:` and `passage:` prefixes. MiniLM uses the raw sentence text in the evaluated pipeline. Retrieved candidates include similarity scores, source page names, sentence identifiers, and sentence text.

4.5 CrossEncoder Reranking

Dense retrieval is efficient, but the highest-similarity sentence is not always the best evidence for a claim. To improve evidence ordering, the system applies a CrossEncoder reranker after dense retrieval. The reranker receives claim-evidence pairs and assigns a relevance score to each candidate sentence. Candidates are then sorted by rerank score, and the top results are passed to the verifier and displayed in the UI.

The reranker improved retrieval quality in the controlled evaluation. Dense retrieval alone achieved Recall@1 of 0.6667 and Recall@5 of 0.9333. After reranking, Recall@1 improved to 0.8133 and Recall@5 improved to 0.9467, while Recall@10 remained 0.9733. This suggests that reranking primarily helps move already-retrieved gold evidence higher in the ranked list rather than finding new evidence outside the dense candidate pool.

4.6 NLI-Based Verification

The verification stage uses a natural language inference model to classify the claim based on retrieved evidence. The verifier is implemented with a CrossEncoder using a DeBERTa-based NLI model. The retrieved evidence passages are combined into a single evidence context and passed to the verifier as the premise, while the claim is passed as the hypothesis. This order is important because NLI models determine whether the premise entails, contradicts, or is neutral toward the hypothesis.

The verifier returns scores for contradiction, entailment, and neutral. These scores are mapped to the final labels `REFUTES`, `SUPPORTS`, and `NOT ENOUGH INFO`. During development, the label order was explicitly inspected from the model configuration to avoid assuming the wrong output index. The final implementation reads the model's label mapping and uses it to identify the correct entailment, contradiction, and neutral indices.

The verifier also supports a score-returning method used by the frontend and evaluation scripts. This allows the system to display not only the predicted label, but also the verifier's score breakdown. These scores support the interpretability goal of the project by helping users understand whether a prediction was strongly supported or uncertain.

4.7 Split Evaluation Pipeline

A key implementation decision was to split the evaluation pipeline into multiple scripts. Initially, retrieval, reranking, and verification were run in a single process. However, loading FAISS, SentenceTransformer models, the reranker CrossEncoder, and the verifier CrossEncoder together caused local native-library crashes on macOS. To make the system stable and reproducible, the final evaluation pipeline separates these stages.

The official evaluation workflow is:

1. Build or reuse the targeted FEVER subset.

2. Build or reuse the FAISS index.
3. Save dense retrieval candidates.
4. Rerank saved candidates.
5. Evaluate the verifier from saved reranked outputs.
6. Run error analysis on incorrect predictions.

This design has two benefits. First, it avoids local stability issues by preventing all models from being loaded in the same Python process. Second, it makes the pipeline easier to debug because each stage writes intermediate outputs to disk. For example, reranked evidence is saved before verification, making it possible to inspect whether errors are caused by retrieval or by the verifier.

4.8 Backend Architecture

The backend is implemented with FastAPI and exposes a `/verify` endpoint. The request body includes the claim, retrieval volume, retriever selection, retrieval mode, and whether reranking should be used. The backend returns a structured JSON response containing the claim, predicted label, confidence value, active settings, verifier scores, and retrieved evidence.

The backend supports two retrieval modes. In FEVER mode, the claim is embedded with the selected retriever model and searched against a prebuilt FAISS index over the targeted FEVER subset. The retrieved evidence can then be reranked before being sent to the verifier. In Live Wiki mode, the backend uses the Wikipedia API to search for relevant pages, fetch page extracts, split them into candidate evidence sentences, and optionally rerank them. The same verifier is then used to classify the claim based on the retrieved evidence.

Live Wiki mode includes a simple in-memory page cache and a delay between page requests to reduce repeated API calls and avoid excessive request frequency. This mode is useful for interactive demonstration because it allows users to test claims beyond the static FEVER subset. However, the reported quantitative metrics are based only on FEVER mode because FEVER provides fixed labels and gold evidence annotations.

4.9 Frontend Architecture

The frontend is implemented as a React web application. It provides a main verification page, settings page, analytics dashboard, and tutorial page. The frontend sends verification requests to the FastAPI backend and renders the returned label, confidence score, verifier scores, and evidence trail.

The settings page allows users to adjust retrieval volume, toggle CrossEncoder reranking, choose available retriever configurations, and switch between FEVER Gold and Live Wiki retrieval. These controls support the project's central goal of making claim verification configurable and interpretable. The analytics page displays retrieval and verification metrics, while the tutorial page explains the pipeline stages from claim input through dense retrieval, reranking, and NLI-based verification.

Overall, the frontend and backend follow a client-server architecture:

`React Frontend` $\rightarrow$ `FastAPI Backend` $\rightarrow$ `Retrieval/Reranking/Verification Pipeline`

This separation allows the UI to remain independent from the retrieval and verification implementation. It also makes the system easier to extend with new retrievers, new evidence sources, or improved verifier models in future work.

5 GenAI Development Process

A major component of this project was not only building the claim verification system itself, but also integrating generative AI tools into the development workflow. We primarily used ChatGPT Pro (GPT-5.5) and Gemini 2.5 Pro for different aspects of development. ChatGPT was used mainly for backend pipeline design, debugging, evaluation scripting, README organization, and technical writeup support, while Gemini was used more heavily during frontend development to rapidly prototype and refine the React-based user interface. This division reflected the strengths of each system: ChatGPT was particularly effective for reasoning through multi-step Python pipeline issues and retrieval logic, whereas Gemini was useful for iterating on visual layouts and UI styling.

Although tools such as Codex were considered, we ultimately chose not to use autonomous coding agents because they were more difficult to control in a collaborative environment where multiple developers were working simultaneously. Instead, we maintained a more traditional software engineering workflow using Git branches, pull requests, and manual merges as new features were developed. This approach provided greater transparency, reproducibility, and coordination while still allowing generative AI systems to accelerate implementation and debugging throughout the project.

5.1 Workflow

Our development process was iterative. We began by implementing the core FEVER data pipeline: loading raw JSONL files, normalizing evidence annotations, processing the Wikipedia dump into sentence-level entries, validating that FEVER gold evidence matched the processed corpus, and building a targeted retrieval subset. ChatGPT was used heavily during this stage to help break the project into small, testable scripts. Instead of attempting to build the full RAG system at once, the project was decomposed into separate modules for loading, preprocessing, retrieval, reranking, verification, and evaluation.

As the project grew, we continued using GenAI tools to reason through implementation choices and debugging steps. For example, ChatGPT helped identify that early low Recall@K results were partly caused by evaluating examples whose evidence pages were not included in the targeted index. It also helped diagnose issues with the natural language inference verifier, including the need to pass inputs in the correct NLI order: evidence as the premise and claim as the hypothesis. These corrections substantially improved the validity of the pipeline.

On the frontend side, Gemini was used to help create the Verified.AI interface, including the verification page, evidence trail, settings page, analytics page, and tutorial page. The frontend was designed as a polished research demo rather than a minimal form interface. To support collaboration between models, ChatGPT was often used to generate and refine prompts for Gemini. Because the backend pipeline was already being developed separately in Python and FastAPI, prompts explicitly instructed Gemini to generate only frontend components using mock data and placeholder state rather than attempting to create its own backend infrastructure. This separation was important because Gemini would otherwise attempt to generate additional backend logic that would later be difficult to integrate cleanly with the existing pipeline. By constraining Gemini to frontend-only development, the React interface could later be connected systematically to the FastAPI backend so that user claims, retrieval settings, and evidence results could be displayed dynamically.

5.2 Where GenAI Helped

GenAI assistance was most effective when the task was specific and modular. ChatGPT was especially useful for designing the sequence of backend scripts, explaining why each stage was necessary,

and helping write reusable code for file loading, evidence normalization, FAISS indexing, retrieval evaluation, and error analysis. It was also helpful for interpreting output logs and deciding what to test next. For example, when the pipeline produced segmentation faults on macOS, ChatGPT helped narrow the source of the failure by isolating FAISS, SentenceTransformer, reranker, and verifier components into separate scripts.

Another area where GenAI helped was documentation. Once the pipeline stabilized, ChatGPT helped organize the README, clarify which scripts belonged to the official workflow, and distinguish experimental/debug scripts from the final evaluation path. This was important because the project accumulated many scripts during development. Without documentation, it would have been difficult for another developer to know which files were part of the final pipeline and which were used only during debugging.

Gemini was useful for frontend scaffolding and visual design. It helped create a more complete interface with navigation, settings controls, analytics cards, evidence display cards, and a tutorial view. This accelerated UI development and made the final system feel like a usable application rather than only a backend experiment. The visual interface also made the project easier to explain because users could see the claim, predicted label, confidence scores, and retrieved evidence in one place.

5.3 Where Human Debugging Was Necessary

Although GenAI tools accelerated development, several key problems required human judgment and iterative testing. The most significant issues were not simple syntax errors, but interactions between the model pipeline, local environment, and evaluation design. For example, the project encountered repeated native crashes involving FAISS, SentenceTransformer, and CrossEncoder models on macOS. These errors appeared as segmentation faults rather than Python exceptions, so they could not be solved by reading a stack trace alone. We had to repeatedly isolate which component was failing, run scripts independently, and restructure the pipeline to avoid loading too many native ML libraries in one process.

Human debugging was also necessary for evaluation correctness. Early retrieval results were misleading because the targeted index and evaluation set were not always aligned. If the targeted subset was built from one set of FEVER examples but evaluation was run on a larger set, some gold evidence pages were not searchable at all. GenAI assistance helped identify this possibility, but verifying it required carefully checking the data flow, rebuilding the index, and comparing results. Once the targeted subset and evaluation set were aligned, retrieval metrics improved substantially.

The verifier also required careful human inspection. Initially, the NLI model was used with the claim and evidence in the wrong order. Since NLI models expect a premise and a hypothesis, the correct input is retrieved evidence as the premise and the claim as the hypothesis. Reversing this order caused unreasonable predictions. Fixing the input order produced much more sensible outputs. This issue demonstrated that even when code runs successfully, model semantics still need to be checked by the developers.

Additionally,it proved especially important during frontend–backend integration because different generative AI systems were used for different parts of the project. Since neither model had full project-wide context or awareness of the other model's outputs, integration frequently required architectural decisions by the developers. For example, frontend components initially relied on mock data structures and placeholder APIs that later needed to be aligned with the actual FastAPI response formats, retrieval settings, and verifier outputs implemented in the backend. Human intervention was therefore necessary to standardize API contracts, resolve state management inconsistencies, reconcile differing assumptions between generated codebases, and ensure that the

independently generated frontend and backend systems operated cohesively as a single application.

Finally, human judgment was necessary to frame the project honestly. The system is strongest on FEVER-style claims where relevant Wikipedia evidence exists in the selected corpus. It is less reliable as a general-purpose fact checker for arbitrary claims. For that reason, we positioned Verified.AI as a transparent and configurable verification workbench rather than a complete replacement for search engines or commercial AI assistants. This framing was important for accurately communicating both the strengths and limitations of the system.

5.4 Takeaways

The main lesson from using GenAI in this project is that AI assistance is most valuable when paired with clear human direction. ChatGPT and Gemini were effective at generating code, proposing pipeline structures, explaining errors, and accelerating frontend development. However, they did not remove the need for systematic debugging, evaluation design, or architectural judgment. The strongest results came from using GenAI to speed up implementation while still manually testing each stage and verifying that the system behaved correctly.

Another takeaway is that modularity matters when developing AI-assisted systems. Because the project was split into separate retrieval, reranking, verification, and analysis stages, we were able to isolate failures and preserve a stable final workflow. This modular structure also made the system easier to explain in the UI and final report. Overall, GenAI acted as a strong development accelerator, but the final project still required human oversight to ensure correctness, stability, and honest evaluation.

6 Evaluation

The system was evaluated using a controlled FEVER-based targeted subset. The goal of evaluation was to measure both retrieval quality and final claim classification performance. Because Verified.AI is designed as a configurable claim-verification workbench, the evaluation also compares different retrieval settings rather than only reporting a single final score.

The targeted evaluation subset was built from the first 100 FEVER examples. Examples without usable gold evidence were removed from metric computation, leaving 75 evaluated examples. The reported results should therefore be interpreted as a controlled FEVER Gold evaluation rather than a full open-domain benchmark. This setup allowed us to directly compare retrieved evidence against FEVER's gold sentence-level annotations.

6.1 Retrieval Metrics

Retrieval performance was measured using Recall@K. For each claim, the system retrieved a ranked list of evidence sentences. A retrieval was counted as successful if at least one gold evidence sentence appeared in the top K retrieved results. We report Recall@1, Recall@5, and Recall@10 to measure how often the correct evidence appears near the top of the ranked output.

Two retrieval configurations were evaluated. The first used dense retrieval only, where the claim embedding was searched against the FAISS index and the top-ranked evidence sentences were returned directly. The second used dense retrieval followed by CrossEncoder reranking. In this setting, the dense retriever first retrieved candidate evidence sentences, and the reranker reordered those candidates based on claim-evidence relevance.

Table 1: Retrieval performance on the targeted FEVER evaluation subset.

Pipeline	Recall@1	Recall@5	Recall@10
Dense Retrieval	0.6667	0.9333	0.9733
Dense Retrieval + Reranker	0.8133	0.9467	0.9733

The reranker improved Recall@1 from 0.6667 to 0.8133 and Recall@5 from 0.9333 to 0.9467. Recall@10 remained unchanged at 0.9733. This suggests that the dense retriever was usually able to include the correct evidence somewhere in the candidate list, while the reranker helped move correct evidence higher in the ranking. In other words, the reranker improved ranking precision more than broad evidence coverage.

6.2 Final Pipeline Accuracy

After retrieval and reranking, the top retrieved evidence passages were passed to the NLI verifier. The verifier classified each claim as `SUPPORTS`, `REFUTES`, or `NOT ENOUGH INFO`. The final pipeline accuracy measures whether the predicted label matched the FEVER gold label.

Table 2: Final claim verification accuracy on the targeted FEVER evaluation subset.

Pipeline	Accuracy
Dense Retrieval + Reranker + Verifier	0.7733

The full pipeline achieved an accuracy of 0.7733 over 75 evaluated examples. This result shows that the system can successfully combine retrieval, reranking, and NLI-based verification for many FEVER-style claims. However, the accuracy is lower than the retrieval Recall@5 score, which indicates that finding the correct evidence is not always sufficient. The verifier must still correctly interpret the evidence and map it to the appropriate FEVER label.

A preliminary evaluation of the Live Wiki mode showed lower overall accuracy (0.43) compared to the standard FEVER gold-evidence evaluation setting. This result is expected because the live pipeline introduces an additional retrieval challenge: the system must first locate relevant evidence from Wikipedia before verification can occur. In contrast, the FEVER benchmark evaluation often provides gold evidence directly, reducing the difficulty of the task and isolating the verifier from retrieval errors.

However, despite the lower quantitative accuracy, the live Wikipedia mode frequently produced evidence that appeared more semantically relevant and intuitive from a human perspective than the FEVER gold evidence itself. In many cases, the top-ranked live retrieval results contained cleaner summaries, more direct factual statements, or broader contextual explanations that were easier for users to interpret due to the large size of the evidence pool. By comparison, FEVER gold evidence is often highly specific, fragmented, or narrowly constructed around benchmark annotations rather than natural human information-seeking behavior.

While FEVER gold evidence provides a standardized method for measuring retrieval and verification performance, live Wikipedia retrieval may better reflect how users actually expect evidence to be surfaced in practical fact-checking systems.

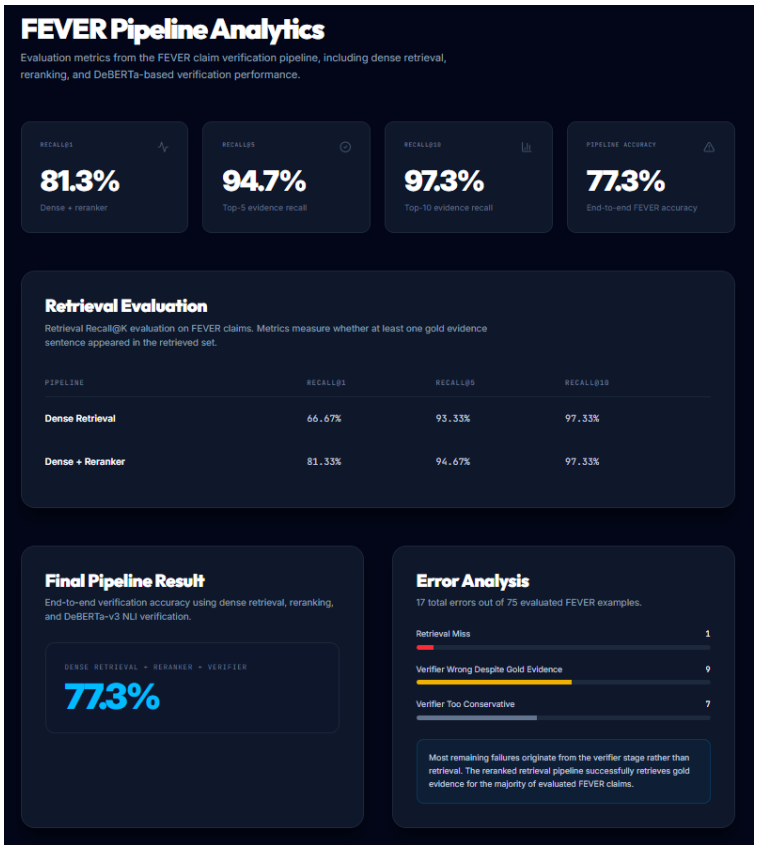

Figure 4: Analytics dashboard summarizes retrieval and verification performance using Recall@K metrics, end-to-end pipeline accuracy, and error analysis visualizations.

6.3 Error Analysis

To better understand the remaining failures, we performed an error analysis on the final reranked pipeline outputs. The system produced 17 errors out of 75 evaluated examples. Each error was categorized based on whether the gold evidence appeared in the retrieved top-5 evidence and how the verifier behaved.

Table 3: Error categories for the final reranked pipeline.

Error Type	Count
Retrieval miss	1
Verifier wrong despite gold evidence	9
Verifier too conservative	7

Only one error was caused by a retrieval miss, meaning that the correct gold evidence did not appear in the top-5 retrieved passages. The remaining errors were verifier-related. In 9 cases, the verifier predicted the wrong label even though gold evidence appeared in the retrieved evidence. In 7 cases, the verifier was too conservative and predicted `NOT ENOUGH INFO` even though the gold label was either `SUPPORTS` or `REFUTES`.

This analysis suggests that retrieval is no longer the main bottleneck in the controlled FEVER evaluation. The system is usually able to retrieve relevant evidence, especially after reranking. Most

remaining failures occur because the verifier struggles with indirect wording, multi-hop reasoning, or evidence that requires connecting multiple facts. This finding motivates future work on stronger verification models, FEVER-specific fine-tuning, and explicit multi-evidence reasoning.

6.4 Interpretation of Results

The evaluation results show that Verified.AI is effective as a controlled, evidence-grounded verification workbench. The retrieval component performs strongly on FEVER-style claims, especially when reranking is enabled. The final accuracy is lower than the retrieval score because label prediction requires more than evidence retrieval; it requires correctly reasoning over the retrieved evidence.

These results also support the design choice of exposing settings and evidence trails in the UI. Since different configurations affect the ranking and final prediction, users benefit from seeing retrieved evidence, rerank scores, and verifier confidence scores rather than only receiving a final label. The system's value is therefore not only in producing an answer, but in making the behavior of the retrieval-augmented pipeline visible and comparable.

7 Discussion and Limitations

The evaluation results show that Verified.AI is most effective as a transparent and configurable claim-verification workbench. In the controlled FEVER setting, the retrieval stage performed strongly, especially after CrossEncoder reranking. The reranker improved Recall@1 and Recall@5, which indicates that it was useful for moving relevant evidence closer to the top of the evidence trail. This is important from a user-interface perspective because users are more likely to inspect the highest-ranked evidence cards than a long list of retrieved passages.

At the same time, the final pipeline accuracy was lower than the retrieval scores. This gap shows that evidence retrieval and claim verification are related but distinct challenges. Even when the correct gold evidence appears in the retrieved results, the verifier may still choose the wrong label or return `NOT ENOUGH INFO`. The error analysis supports this interpretation: only one error was caused by a retrieval miss, while most errors came from verifier mistakes or overly conservative verifier behavior. This suggests that future improvements should focus more on verification and reasoning than on simply retrieving more evidence.

7.1 Targeted Subset Limitation

The most important limitation of the reported evaluation is that it uses a targeted FEVER subset rather than the full FEVER Wikipedia corpus. The targeted subset was built by collecting Wikipedia pages that appeared in the gold evidence annotations for the selected FEVER examples. This made local evaluation computationally manageable and allowed us to test whether the system could retrieve the correct sentence when the relevant evidence page was present in the candidate corpus.

However, this setup is easier than full open-domain retrieval. In a full FEVER evaluation, the retriever would need to search across millions of Wikipedia sentences without already knowing which pages are relevant. Because of this, the reported Recall@K and accuracy values should not be interpreted as full FEVER benchmark results. They demonstrate performance in a controlled setting, not general open-domain fact-checking performance.

This design choice was still useful for the project because it allowed us to isolate pipeline behavior. By reducing the page-retrieval challenge, we could focus on sentence retrieval, reranking,

verification, UI integration, and error analysis. The targeted subset therefore served as a practical development and evaluation strategy, but scaling to the full corpus remains future work.

7.2 FEVER-Style Claim Limitation

Verified.AI performs best on claims that resemble FEVER examples. FEVER claims are designed around Wikipedia evidence and have known sentence-level annotations. Arbitrary user-entered claims may be harder to verify because they can be vague, current, opinion-based, or require information outside Wikipedia. For example, a claim about a recent event may not be covered in the static FEVER snapshot. Similarly, a claim requiring domain-specific expertise or non-Wikipedia sources may not be handled well by the current system.

The Live Wiki mode partially addresses this limitation by retrieving current Wikipedia evidence through live API requests. However, Live Wiki mode was not evaluated with gold labels in the same way as FEVER Gold mode. It is best understood as an interactive demonstration feature rather than a rigorously benchmarked retrieval setting. This distinction is important: the reported metrics apply to FEVER Gold mode, while Live Wiki mode shows how the system could be extended beyond the static FEVER corpus.

7.3 Verifier Limitations

The verifier is the main remaining bottleneck. The system uses a DeBERTa-based natural language inference model to classify whether the retrieved evidence entails, contradicts, or is neutral toward the claim. This works well for many direct claims, but it struggles when the evidence requires implicit reasoning, multiple supporting sentences, or entity-level inference.

Some errors occur when the verifier is too conservative. In these cases, relevant evidence appears in the retrieved top results, but the verifier predicts `NOT ENOUGH INFO`. This often happens when the evidence is related but does not explicitly use the same wording as the claim. Other errors occur when the verifier predicts the wrong polarity despite gold evidence being present. These cases suggest that simply passing concatenated evidence into an NLI model is not always enough for robust claim verification.

A stronger verifier could be trained or fine-tuned specifically on FEVER claim-evidence pairs. Another improvement would be explicit multi-hop reasoning, where the system identifies which evidence sentences should be combined before classification. The current verifier treats the retrieved evidence as a combined context, but it does not explicitly reason over evidence chains or distinguish which sentence is responsible for which part of the claim.

7.4 Retriever and Model Configuration Limitations

The interface was designed to support multiple retriever configurations, including MiniLM, BGE, and E5. In practice, the reported evaluation uses MiniLM because it was the stable retriever used throughout the controlled FEVER evaluation. Additional retriever options were added to support experimentation, but local model-loading and embedding-generation issues made some configurations less stable in our development environment.

This limitation reinforces the importance of the settings page. Verified.AI exposes retrieval configuration because different models and settings can produce different evidence rankings. However, exposing settings also means that some configurations may require additional setup, stronger hardware, or a more stable deployment environment. For the final evaluated pipeline, we rely on the MiniLM-based configuration to preserve reproducibility.

7.5 Local Stability and Engineering Limitations

A practical limitation of the project was local environment stability. Several components rely on native libraries, including FAISS, PyTorch, sentence-transformers, and CrossEncoder models. During development, running retrieval, reranking, and verification in a single Python process caused segmentation faults on macOS. These were not normal Python exceptions and required restructuring the pipeline.

To address this, the final evaluation workflow was split into separate stages. Dense retrieval saves candidate evidence to disk, reranking loads and reranks those candidates separately, and verification is run from saved outputs. This makes the pipeline more stable and easier to debug, but it is less convenient than a single unified evaluation script. In a production deployment, this issue could be addressed through a Linux-based environment, containerization, or a service architecture where each model component runs in its own process.

7.6 User Interface Limitations

The user interface makes the pipeline more interpretable, but not every displayed value should be interpreted as a calibrated probability. The confidence score is derived from verifier output scores, which are useful for comparison but are not fully calibrated confidence estimates. Similarly, the evidence scores and rerank scores help users understand ranking behavior, but they do not guarantee factual correctness on their own.

The analytics page includes visualizations such as recall indicators and confidence breakdowns. These are useful for demonstrating pipeline behavior, but some analytics elements are best understood as interpretability aids rather than formal evaluation outputs. The formal reported results remain the Recall@K, accuracy, and error analysis values from the controlled FEVER evaluation.

7.7 Overall Discussion

Overall, Verified.AI demonstrates the value of combining retrieval, reranking, verification, and user-facing explanation in a single system. The strongest part of the system is evidence retrieval: in the controlled FEVER setting, the reranked retriever usually places relevant evidence in the top results. The weaker part is final reasoning, where the verifier can still misclassify claims even when useful evidence is available.

This outcome supports the core framing of the project. Verified.AI should not be viewed as a universal replacement for Google Search, commercial AI assistants, or professional fact-checking organizations. Instead, it is best understood as a configurable and interpretable verification workbench. Its value lies in exposing how different retrieval settings, reranking choices, and evidence sources affect the final prediction. This makes the system useful for studying retrieval-augmented verification and for demonstrating the strengths and limitations of evidence-grounded AI systems.

8 Future Work and Conclusion

8.1 Future Work

The most important future direction is scaling beyond the targeted FEVER subset. The current evaluation uses a controlled subset where relevant Wikipedia pages are already included in the candidate corpus. A full-scale version of the system should index the complete FEVER Wikipedia corpus or a much larger Wikipedia snapshot. This would make retrieval more realistic because the system would need to identify both the correct page and the correct sentence from a much larger

evidence space. Running this at scale would likely require a Linux environment, more memory, stronger compute resources, or a hosted vector database.

A second direction is improving the verifier. Error analysis showed that most remaining mistakes were not caused by retrieval misses, but by verifier errors. In several cases, the correct evidence appeared in the retrieved top results, but the verifier either predicted the wrong label or returned `NOT ENOUGH INFO`. Future work could fine-tune the verifier on FEVER claim-evidence pairs so that it better handles the specific wording, evidence structure, and label distribution of the dataset. Another option is to replace the current single-step NLI verifier with a model that explicitly reasons over multiple evidence sentences.

Multi-hop reasoning is another important area for improvement. Some FEVER claims require combining information from multiple sentences or pages. The current implementation concatenates retrieved evidence before verification, but it does not explicitly identify which pieces of evidence support which parts of the claim. A stronger system could first select an evidence set, then reason over that set as a structured chain before producing a label. This would likely improve performance on claims where a single sentence is not enough.

The Live Wiki mode also creates opportunities for future development. Since it retrieves evidence from current Wikipedia pages, it helps move the system beyond the static FEVER snapshot. However, it currently depends on live API requests and can be affected by rate limits, page search quality, and sentence-splitting limitations. Future work could add persistent caching, better entity extraction, improved page ranking, and source filtering. This would make Live Wiki mode more reliable for interactive use.

The user interface could also be extended further. The current interface already supports dynamic claim verification, retrieval mode selection, configurable pipeline settings, evidence visualization, analytics dashboards, and an interactive tutorial system. At present, the analytics dashboard displays static metrics generated from separately executed evaluation pipelines, including FEVER benchmark results and live Wikipedia evaluation statistics. While this provides useful insight into overall system behavior, future versions could make the analytics system fully interactive by connecting it directly to evaluation outputs and runtime experiments.

One particularly useful extension would be support for side-by-side comparison workflows. Users could compare different retriever models, retrieval volumes, or reranking configurations using dynamic graphs and dropdown-based controls. For example, the system could allow users to run the same claim under FEVER gold evidence mode and live Wikipedia retrieval mode simultaneously, then visualize differences in retrieved evidence, ranking quality, verifier confidence scores, and final predictions. Additional extensions could include saved verification sessions, exportable reports, interactive confusion matrices, and longitudinal tracking of retrieval performance across different experimental settings. Since Verified.AI is designed as a retrieval-augmented verification workbench rather than a single-purpose fact-checker, these comparison and visualization features would significantly improve its usefulness for experimentation, debugging, and educational analysis.

Finally, deployment could be improved. During development, some model combinations were unstable in the local macOS environment. A more robust deployment would use containerization or separate backend services for retrieval, reranking, and verification. This would allow each model component to run independently and would reduce the chance of native-library conflicts. It would also make the system easier to scale and maintain.

8.2 Conclusion

This project implemented Verified.AI, a configurable RAG-style claim verification workbench that combines dense retrieval, FAISS indexing, CrossEncoder reranking, and NLI-based verification.

Given a natural language claim, the system retrieves Wikipedia-based evidence, reranks candidate passages, predicts a FEVER-style label, and displays the evidence and verifier scores through a web interface. The final system includes both a backend verification pipeline and a React-based frontend for interacting with claims, settings, evidence trails, analytics, and tutorial content.

The controlled FEVER evaluation showed strong retrieval performance. On the targeted 100-example FEVER evaluation, the dense retrieval plus reranker pipeline achieved Recall@1 of 0.8133, Recall@5 of 0.9467, and Recall@10 of 0.9733. The full retrieval, reranking, and verification pipeline achieved an accuracy of 0.7733 over 75 examples with usable gold evidence. Error analysis showed that only one error came from a retrieval miss, while most remaining errors were caused by verifier mistakes or overly conservative verifier behavior. This indicates that the main bottleneck is no longer evidence retrieval, but reasoning over retrieved evidence.

The project also demonstrates the importance of transparency in claim verification. Rather than hiding the retrieval process behind a single answer, Verified.AI exposes retrieved evidence, similarity scores, rerank scores, verifier scores, and configurable retrieval settings. This makes the system useful not only as a fact-checking demo, but also as an educational and experimental tool for understanding retrieval-augmented AI behavior.

Overall, Verified.AI should be understood as a transparent verification workbench rather than a universal fact-checking replacement. It performs best in controlled FEVER-style settings and provides a strong foundation for exploring how retrieval, reranking, and verification interact. Future work should focus on scaling retrieval, improving verifier reasoning, supporting more reliable live evidence retrieval, and expanding the UI into a richer comparison environment for evidence-grounded AI systems. The current implementation is a locally hosted application rather than a publicly deployed web service. During development and demonstration, the frontend is run locally with `npm run dev`, and the FastAPI backend is run locally with `uvicorn`. This setup is sufficient for project demonstration and local experimentation, while public deployment remains future work.

Repository: `https://github.com/camlyt/CSCI455FinalProject`